PRECONDITIONING AND NUMERICAL STABILITY IN NEURAL NETWORK TRAINING FOR PARAMETRIC PDES

MARKUS BACHMAYR[1], WOLFGANG DAHMEN[1,2], CHENGUANG DUAN[1], AND MATHIAS OSTER[1]

Abstract. In the context of training neural network-based approximations of solutions of parameter-dependent PDEs, we investigate the effect of preconditioning via well-conditioned frame representations of operators and demonstrate a significant improvement on the performance of standard training methods. We also observe that standard representations of preconditioned matrices are insufficient for obtaining numerical stability and propose a generally applicable form of stable representations that enables computations with single- and half-precision floating point numbers without loss of precision.

Keywords. parameter-dependent partial differential equations, preconditioning, finite element frames, neural networks, numerical stability

Mathematics Subject Classification. 41A30, 65D40, 65N55

1. Introduction

1.1. **Problem formulation.** The need for computationally efficient approximations of solutions of parameter-dependent PDEs arises in many different applications. Abstractly, given a family of differential operators $\mathcal{R}_y$ in residual form, parameterized by $y \in Y$, solutions $u(x, y)$ of $\mathcal{R}_y(u(\cdot, y)) = 0$ can be viewed as functions of spatio-temporal variables $x \in \Omega \subset \mathbb{R}^d$ as well as of the parametric variables y ranging over a given parameter domain Y. Here Y can be a subset of a finite-dimensional Euclidean space, $Y \subset \mathbb{R}^p$ with some $p \in \mathbb{N}$, or more generally of an infinite-dimensional sequence or function space. In which sense u (or a derived quantity of interest) is to be approximated over $\Omega \times Y$, is determined by approximating the mapping $y \mapsto u(y) = u(\cdot, y)$ in appropriate norms for functions on Y and Ω. This task, sometimes referred to as "operator learning," can arise both in a context of model reduction (as for the accelerated solution of inverse or optimization problems) and in uncertainty quantification.

Especially for challenging problems that are not amenable to linear model reduction methods such as POD or reduced basis methods, approximations for u based on neural networks are of interest. While in certain approaches such as physics-informed neural networks (PINNs), values of u are approximated by a neural network with input parameters x and y, it can be beneficial to use neural network representations only in the (potentially high-dimensional) y-variable, while the dependence on x is approximated by well-understood classical discretization methods. Assuming $\{\varphi_i\}_{i\in\mathcal{I}}$ with $\#\mathcal{I} < \infty$ to be a suitable set of fixed basis functions on Ω, this leads to hybrid approximations of the form

$$u(x, y) \approx \tilde{u}(x, y; \theta) = \sum_{i\in\mathcal{I}} \mathbf{u}_i(y; \theta)\varphi_i(x), \tag{1.1}$$

where the approximation coefficients $\mathbf{u} = (\mathbf{u}_i)_{i\in\mathcal{I}}$ are jointly represented as neural networks and depend on y as well as on a vector θ of trainable network parameters. Note that the

[1] Institut für Geometrie und Praktische Mathematik, RWTH Aachen University, Im Süsterfeld 2, 52072 Aachen, Germany

[2] University of South Carolina, Mathematics Department, 1523 Greene Street, Columbia, SC 29208, USA

Email address: bachmayr@igpm.rwth-aachen.de, dahmen@igpm.rwth-aachen.de, duan@igpm.rwth-aachen.de, oster@igpm.rwth-aachen.de.

The authors acknowledge funding by Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – project number 442047500/SFB 1481 *Sparsity and Singular Structures*. M.B. acknowledges funding by the European Union (ERC, COCOA, 101170147). W.D. acknowledges funding by National Science Foundation grants FRG DMS-2245097, DMS-2513112, and RTG DMS-2038080.

basis functions φ_i could be standard finite element basis functions or elements of a problem-adapted reduced basis. Representations of the form (1.1) have been used, for example, for problems with finite-dimensional Y in [2, 10, 20, 22, 26], but are also common in the setting of *operator learning* with infinite-dimensional Y [4, 25].

For any method that yields a quantification of the achieved accuracy of approximations with respect to a suitable norm, it is important to decide in which (weak) sense the equation $\mathcal{R}_y(u(\cdot, y)) = 0$ is required to hold. For a proper weak formulation, one should have $\mathcal{R}_y \colon U_y \to V_y'$ with Hilbert spaces U_y, V_y for each y, postponig for a moment further comments on the choice of the trial- and test-spaces U_y, V_y. A commonly used way to obtain the approximation $\tilde{u}$ to $y \mapsto u(y) \in U_y$ is then to employ *empirical risk minimization*

$$\min_\theta \sum_{k=1}^K \|u(y^k) - \tilde{u}(y^k;\theta)\|_{U_{y^k}}^2, \tag{1.2}$$

see, e.g. [20, 26], where for the sake of computational convenience, the norms $\|\cdot\|_{U_y}$ are sometimes replaced by L_2-norms. Viewing the parameters as random variables with probability distribution μ and assuming the y^k, $k = 1, \dots, K$, to be i.i.d. draws from μ, this corresponds to approximately minimizing a Monte Carlo approximation of the expectation $\mathbb{E}_{y\sim\mu}\big[\|u(y) - \tilde{u}(y^k;\theta)\|_{U_{y^k}}^2\big]$. If the norms $\|\cdot\|_{U_y}$ are uniformly equivalent over Y, the expectation is a standard Bochner norm. If not, under mild conditions on the scalar products, inducing $\|\cdot\|_{U_y}$, the above expectation is still well-defined as a so-called direct integral, see [2].

Alternatively, to mitigate the computational cost of a large number of high-fidelity training samples $u(y^k)$, it can also be advantageous to instead use *residual losses*. To infer from a residual the accuracy of the approximation it is essential in which norm the residual is measured. As shown in [2], under the above assumptions on the mapping properties of $\mathcal{R}_y$, a proper choice is $\|\mathcal{R}_y(w)\|_{V_y'}$ (the dual test norm), because if the underlying weak formulations are stable one has $\|\mathcal{R}_y(w)\|_{V_y'} \eqsim \|w\|_{U_y}$, a property that we refer to as *variational correctness*. Hence the size of the loss is a reliable and efficient a posteriori error bound. Then (1.2) can be replaced by

$$\min_\theta \sum_{k=1}^K \big\|\mathcal{R}_{y^k}\big(\tilde{u}(y^k;\theta)\big)\big\|_{V_{y^k}'}^2. \tag{1.3}$$

The issue that we address in this paper concerns both types of optimization problems (1.2) and (2.5). The quality of corresponding numerical results depends crucially on the performance of optimization schemes used for the loss minimization. While the nonlinear structure of neural networks poses an unavoidable obstacle, the separation of spatial and parametric variables, facilitated by the hybrid format (1.1), nevertheless offers substantial advantages. A central point in what follows is that each summand in the objective is a quadratic form that offers room for formulating the objective such that it becomes favorable for descent strategies. An intrinsic obstruction lies in the fact that the norms on U_y and V_y are typically induced by differential operators, for example by the Laplacian when $U_y = H_0^1(\Omega)$. We explain later below in more detail that both types of loss (1.2) and (2.5) exhibit similar features with regard to the dependence on differential operators, opening the door to exploiting *preconditioning* techniques for removing PDE-related ill-conditioning from the optimization problems.

1.2. **Choice of basis, preconditioning, and performance of optimization schemes.**

The most basic and well-understood scenario is $\mathcal{R}_y v = A_y v - f$, where $A_y \colon U_y \to V_y'$ is a linear operator and $f \in V_y'$. This corresponds to a weak formulation

$$\langle A_y u(y), v\rangle = f(v), \quad \forall\, v \in V_y,\ y \in Y.$$

Variational correctness in the above sense is then equivalent to the well-posedness of this weak formulation (that is, the conditions in the Babuška-Nečas theorem hold), which in turn means that $A_y : U_y \to V_y'$ is a norm isomorphism.

For simplicity of exposition, we assume that the spaces U_y, V_y agree for all y with equivalent norms, so that we can suppress dependence on y in corresponding notations. Note that then the problems

$$\min_{w\in U} \|A_y w - f\|_{V'}^2 \quad \text{or} \quad \min_{w\in U} \|u(y) - w\|_U^2 \tag{1.4}$$

are *well-conditioned* when posed in V', U, respectively, so that descent methods in U or V' would converge rapidly. However, as soon as these problems are posed over some finite dimensional trial space

$$U_{\mathcal{I}} = \operatorname{span} \Phi, \quad \Phi = \{\varphi_i : i \in \mathcal{I}\},$$

the objective becomes a quadratic form over $\mathbb{R}^{\#\mathcal{I}}$ whose numerical conditioning now depends on the choice of the representation system Φ in the same way as it affects the performance of iterative solvers for resulting linear systems, namely through the condition number of the Gramian $(\langle\varphi_i, \varphi_{i'}\rangle_U)_{i,i'\in\mathcal{I}}$, associated to Φ. Obviously, an ideal choice of Φ would be a subset of a *U-orthonormal* system that would exactly preserve the properties of the problem on (1.4) and lead to resulting condition numbers uniformly bounded in $\#\mathcal{I}$. While suitable reduced basis systems may have this property, this is not true for a generic finite element basis, which is the primary concern in what follows.

We remark that if for $V' \neq V$, the residual loss $\|A_y w - f\|_{V'}^2$ is replaced by the simpler one $\|A_y w - f\|_{L_2}^2$ as in PINN, A_y is not necessarily an isomorphism onto L_2, so that even the continuous problem on U is ill-posed, adding to the difficulties incurred by subsequent discretizations.

Preconditioning the Gramian $(\langle\varphi_i, \varphi_{i'}\rangle_U)_{i,i'\in\mathcal{I}}$ is of obvious importance for reducing the number of descent steps in a minimization and thus a first subject of this paper. However, there is a second issue that turns out to play a decisive role, especially in connection with optimization, or more precisely, when dealing with nonlinear parameterizations for the solution coefficients $\mathbf{u}$, namely the possible *loss of numerical precision*, due to the standard application of a preconditioner.

It needs to be noted that this is indeed a separate issue from obtaining a well-conditioned problem for the coefficients $\mathbf{u}$; see, for example, related discussions in [3,28]. We will demonstrate how an appropriate combination of a variational formulation for the family of PDEs and a representation systems Φ for the spatial discretizations affects several points in the solution process: the *norms* with respect to which errors can be controlled, effective *initializations* and the *converence* of optimization schemes, as well as limitations on attainable precision in floating point arithmetic.

While difficulties in reducing the objective can to some degree be mitigated by more sophisticated optimization methods, for example the Gauss-Newton (in this context also known as *natural gradient descent*), such techniques do not address the loss of significant digits in the result due to the action of ill-conditioned matrices. Especially when using lower-precision arithmetic for using neural network training, this will be seen to severely limit the attainable accuracies.

1.3. **Stable representations and relation to prior works.** As we demonstrate theoretically and quantify numerically in the following sections, well-posed variational formulations combined with well-conditioned frame representations can lead to greatly improved convergence of optimization methods. However, we stress that preconditioning by itself does not automatically lead to satisfactory results when using lower machine precision, such as 32-bit (single-precision) or even 16-bit (half-precision) floating point numbers. As an additional ingredient, recovering full-precision results even for low machine precision requires *well-conditioned decompositions* of discretization matrices, which we refer to as *stable representations* and explain below in more detail.

The approach proposed in this paper can be seen in the context of several prior works on similar issues, albeit from different perspectives and for varying applications. In the context of iteratively solving large linear systems, it is well understood that a good preconditioner may significantly speed up the convergence of iterative schemes, but may fail to achieve high accuracy in the numerical solution. The reason is that the preconditioned matrix may be given as a product of several matrices which themselves are still ill-conditioned; see, for example, the references in [28]. For systems arising from discretizing an operator equation, this issue has been addressed systematically by so-called *full operator preconditioning* (FOP) in [28]. The key idea is to first reformulate a given operator equation on the continuous level by means of multiplying the original operator from the left and the right by suitable operators in such a way that the resulting product becomes well-conditioned on spaces for which suitable trial and test bases are available (such as L_2-spaces).

The prospective gain lies in avoiding separate discretizations of ill-conditioned factors. In summary, to warrant both rapid convergence of the preconditioned iteration as well as high numerical accuracy, the condition of the transformed operator equation as well as the condition of trial basis should be moderate or small. The present paper shares the central objective of avoiding adverse effects of ill-conditioned factors in a preconditioned matrix. However, an important distinction is that for the type of problems considered here, we do not need to first reformulate an operator equation on the continuous level. Instead, it suffices to use a non-standard factorization of standard preconditioned matrix representation. To the best of our knowledge, this fact has been observed first in the context of multilevel tensor representations in [3], which is the main source of inspiration also for the present paper. Moreover, the role of frames in the present context can be related to "suitability" of trial bases in the FOP context and hence is a task to be addressed in either approach.

The main mechanism used here has subsequently also been exploited in the quite different context of quantum algorithms for finite element systems [16]. There, a central role is played by so-called block encodings of matrices revolving around projections on unitary representations under normalization constraints. The complexity of such algorithms, determined by the number of iterations, is governed by the effective condition of such block encodings, which again depends on factorizations with well-conditioned factors. Here, ill-conditioned factors cause a loss in precision due to projection and normalization constraints.

From the point of view of only speeding up convergence, there are numerous other related works, most notably in the context of machine learning and neural operator learning. For instance, for neural network approximations of parameter-dependent problems, preconditioning also enters the discussion in [22]. In this case, left preconditioning is combined with a least squares formulation to obtain a well-conditioned problem. However, this amounts to using an L_2-space as trial space, so that errors can only be controlled with respect to this norm. Under the flag of natural gradient flows in the context of supervised learning, the approximate inversion of the Gramian of the weight gradient $\nabla_\theta u(\cdot;\theta)$ can be viewed as preconditioning; see, for example, [7,9,13,27]. The scalable solution of the arising subproblems, however, again hinges on suitable preconditioning for these Gramians.

In the present paper, we show that also standard multilevel finite element preconditioners can be brought into alternative matrix forms – computable with the same ease and efficiency as their standard counterparts – that lead to numerically stable methods without loss of accuracy even for low machine precision. Hence, approximating the spectrum of Hermitian squares of operators as in [28] is not required for avoiding a loss of precision. As shown below, it can instead suffice to re-express standard preconditioned matrices in terms of slightly different alternative representations.

The code used for the numerical experiments given in this paper can be found in [1].

2. Norm stability of representation systems and condition numbers

We first discuss the influence of the representation system $\Phi = \{\varphi_i : i \in \mathcal{I}\}$ in (1.1) on the performance of optimization methods for (1.2) and (2.5). For simplicity of exposition, we focus on Galerkin discretizations of elliptic problems. In these problems, the identical trial

and test spaces can be chosen to be independent of the parameter and are denoted by U. The basic principles apply more generally to Petrov-Galerkin-type methods, where trial and test spaces differ and may depend (even as sets) on y.

In order to explain the basic concepts, we use the classical model problem

$$-\operatorname{div}\big(a_y \nabla u(y)\big) = f \quad \text{in } \Omega, \qquad u|_{\partial\Omega} = 0, \tag{2.1}$$

in appropriate weak formulation, where the diffusion field a_y is bounded and strictly positive uniformly with respect to $y \in Y$.

2.1. **Examplary variational formulations.** In what follows, we make use of a common structure in weak formulations. We assume that the problem can be written in the form: find $u\colon Y \mapsto U$ such that

$$\langle \mathcal{D}u(y), \mathcal{D}v\rangle_y = \langle \ell, v\rangle \quad \text{for all } v \in U,\ y \in Y. \tag{2.2}$$

Here we require the y-independent mapping $\mathcal{D}\colon U \to H$, with $H = \big(L_2(\Omega)\big)^m$ for appropriate $m \in \mathbb{N}$, to be bounded and injective such that $\mathcal{D}\colon U \to \operatorname{ran}(\mathcal{D}) \subset H$ is an isomorphism. Moreover, we assume $\langle\cdot,\cdot\rangle_y$ to be a bilinear form on H that may depend on y. We are interested in equivalently expressing (2.2) as a quadratic minimization problem.

Example 1. We assume that for $a_y \in L_\infty(\Omega)$ there exist constants a_1, a_∞ such that $0 < a_1 \le a_y(x) \le a_\infty < \infty$ for all $x \in \Omega$ and $y \in Y$. Then for $U = H^1_0(\Omega)$, $\mathcal{D} = \nabla$, $m = d$ and $\langle\cdot,\cdot\rangle_y = \langle a_y\cdot,\cdot\rangle_{(L_2)^m}$, the above assumptions are valid and the solution to (2.2) is characterized as the unique minimizer of the standard energy minimization

$$\min_{u\in H^1_0(\Omega)} \big\{ \tfrac{1}{2}\langle a_y\nabla u, \nabla u\rangle_{L_2} - \langle f, u\rangle \big\}. \tag{E}$$

Moreover, we have

$$c_U\|u\|_U^2 \le \langle \mathcal{D}u, \mathcal{D}u\rangle_y \le C_U\|u\|_U^2 \quad \forall\, u \in U,\ y \in Y \tag{2.3}$$

with constants $0 < c_U \le C_U < \infty$ depending only on Ω, a_1, and a_∞.

Example 2. Under the assumptions of Example 1, introducing $\sigma := a_y\nabla u$ as an auxiliary variable, the solution to (E) can also be obtained via the *first-order system least squares* (FOSLS) formulation

$$\min_{\substack{u\in H^1_0(\Omega)\\ \sigma\in H(\operatorname{div};\Omega)}} \Big\{ \|\operatorname{div}\sigma - f\|_{L_2}^2 + \|a_y^{-1}\sigma - \nabla u\|_{L_2}^2 \Big\}. \tag{LS}$$

In this case, $U = H^1_0(\Omega)\times H(\operatorname{div};\Omega)$, $m = 2d+1$, and (2.3) holds for

$$\mathcal{D}(v,\tau) = \begin{pmatrix} \nabla v \\ \tau \\ \operatorname{div}\tau \end{pmatrix}, \quad \big\langle (g,\tau,p), (h,\rho,q)\big\rangle_y = \langle p, q\rangle_{L^2} + \langle a_y^{-1}\tau - g, a_y^{-1}\rho - h\rangle_{(L^2)^d}.$$

In fact, by [5, Thm. 5.15] (which was originally obtained in [8]) there exist $c_U, C_U > 0$ depending on a_1 and a_∞ in Example 1 such that

$$\big\langle (\nabla u, \tau, \operatorname{div}(\tau)), (\nabla u, \tau, \operatorname{div}(\tau))\big\rangle_y \ge c_U\big(\|\tau\|_{H(\operatorname{div})}^2 + \|u\|_{H^1_0(\Omega)}^2\big).$$

Similarly, by the upper bound on a_y and the Cauchy-Schwarz inequality, we have

$$\big\langle (\nabla u, \tau, \operatorname{div}(\tau)), (\nabla v, \sigma, \operatorname{div}(\sigma))\big\rangle_y \le C_U\big(\|\operatorname{div}(\tau)\|\|\operatorname{div}(\sigma)\| + (\|\tau\| + \|\nabla u\|)(\|\sigma\| + \|\nabla v\|)\big),$$

see [2, 5, 8] for further details.

One advantage of (LS) is that information on the flux σ, which is often of primary interest, is obtained without post-processing (which may diminish accuracy) of the solution u to (E). In the context of variationally correct residual regression [2], the following observation is essential: while substituting an approximate solution into the energy functional (E) does not show its accuracy, the formulation (LS) has the advantage of directly giving a quantification

of the total solution error. Indeed, for any $v \in H^1_0(\Omega)$, $\tau \in H(\mathrm{div};\Omega)$, for the error with respect to the solution (u,σ) of (LS), we have

$$\|u-v\|^2_{H^1_0} + \|\sigma-\tau\|^2_{H(\mathrm{div})} \eqsim \|\mathrm{div}\,\sigma - f\|^2_{L_2} + \|a_y^{-1}\sigma - \nabla u\|^2_{L_2}, \tag{2.4}$$

with proportionality constrants depending only on a_1, a_∞, and Ω.

Note that ℓ in (2.2) may generally also depend on the parameter y. We dispense here with including this case since it is not essential for subsequent considerations. Least squares formulations similar to (LS) exist also for parabolic problems [19] and for acoustic wave equations [18], and we expect that the results given here can be transferred directly to FOSLS formulations of such problems.

2.2. **Riesz basis and frame representations.** With a choice of Φ, residual regression for the problems (E) and (LS) can both be written in the form

$$\min_\theta \sum_{k=1}^K \big(\langle \mathbf{A}_{y^k}\mathbf{u}(y^k;\theta), \mathbf{u}(y^k;\theta)\rangle_{\ell_2} - \langle \mathbf{f}, \mathbf{u}(y^k;\theta)\rangle_{\ell_2}\big), \tag{2.5}$$

based on (2.2), where

$$\mathbf{A}_y = \big(\langle \mathcal{D}\varphi_{i'}, \mathcal{D}\varphi_i\rangle_y\big)_{i,i'\in\mathcal{I}}, \quad \mathbf{f} = \big(\langle \ell, \varphi_i\rangle\big)_{i\in\mathcal{I}}. \tag{2.6}$$

Using regression for the solution as in (1.2) leads to a problem of the same structure, except that $\mathbf{A}_y$ does not depend on y because the inner product providing the entries of the Gramian can be chosen to be independent of y.

It is well known that standard finite element basis functions lead to ill-conditioned discretization matrices $\mathbf{A}_y$, since they form well-conditioned bases in L_2 but generally not in U. This problem can be circumvented by using Riesz bases Φ (see, for example, [11, 12]), that is, φ_i for $i \in \mathcal{I}$ are required to satisfy

$$c_\Phi \|\mathbf{v}\|_{\ell_2} \le \Big\|\sum_{i\in\mathcal{I}} \mathbf{v}_i\varphi_i\Big\|_U \le C_\Phi \|\mathbf{v}\|_{\ell_2}. \tag{2.7}$$

Examples of Riesz bases for Sobolev spaces U are wavelet or Fourier bases, as constructed in [14,15] on polygonal domains. However, the construction of such bases becomes technically more demanding with increasing geometric complexity of the domain.

The Riesz basis property means that the *frame operator*

$$F\colon \ell_2(\mathcal{I}) \to U, \quad \mathbf{v} \mapsto \sum_{i\in\mathcal{I}} \mathbf{v}_i\varphi_i \tag{2.8}$$

is an isomorphism from $\ell_2(\mathcal{I})$ to $U_\Phi = \overline{\mathrm{span}\,\Phi}$ and $\|F\|_{\ell_2\to U} \le C_\Phi$. This condition can be relaxed by instead allowing Φ to be *overcomplete*, but such that F is still an isomorphism from $\ker(F)^\perp$ to U_Φ, or equivalently, that $F'\colon U'_\Phi \to \mathrm{ran}(F') \subset \ell_2(\mathcal{I})$ is an isomorphism. As one can show, this in turn is equivalent to the *frame property*

$$c_\Phi^2 \|h\|^2_{U'_\Phi} \le \sum_{i\in\mathcal{I}} |h(\varphi_i)|^2 \le C_\Phi^2 \|h\|^2_{U'_\Phi} \quad \text{for all } h \in U'_\Phi, \tag{2.9}$$

which we adopt here as a definition of a frame Φ for $U_\Phi \subset U$.

We refer to c_Φ, C_Φ as corresponding frame constants. Such frames are much easier to construct and play a pivotal role in the theory of *additive Schwarz methods* addressed below; for an overview, we refer to [29].

2.3. **Multilevel frames.** One way to construct stable frames is based on multilevel structures in finite element methods. Let $W_j \subset H^1(\Omega)$ for $j \in \mathbb{N}$ be a hierarchy of finite element subspaces such that $W_j \subset W_{j+1}$ with standard H^1-normalized Lagrange basis $\{\varphi_{j,k}\colon k \in \mathcal{I}_j\}$ for each W_j. The following is a classical result in the context of additive Schwarz methods; see, for example, [23, 29, 31].

Theorem 3. *Let $\|\varphi_{j,k}\|_{H^1(\Omega)} \eqsim 1$. Then $\{\varphi_{j,k}\colon j = 1,\dots,J,\ k \in \mathcal{I}_j\}$ is a frame for $W_J \subset H^1(\Omega)$ with frame constants independent of J.*

The stiffness matrix with respect to the finest discretization level J reads

$$\mathbf{A}_y = \big(\langle \mathcal{D}\varphi_{J,k}, \mathcal{D}\varphi_{J,k'}\rangle_y\big)_{k,k'\in\mathcal{I}_J}. \tag{2.10}$$

Let $N = \#\mathcal{I}_J$ and $\hat{N} = \sum_{j=1}^{J} \#\mathcal{I}_j$. We denote by $\mathbf{H}$ the mapping that takes the frame expansion coefficients into the coefficients with respect to the basis of W_J, that is,

$$\mathbf{H}\colon \mathbb{R}^{\hat{N}} \to \mathbb{R}^N, \quad \mathbf{w} = (\mathbf{w}_{j,k})_{\substack{j=1,\dots,J\\ k\in\mathcal{I}_j}} \mapsto \mathbf{v} = (\mathbf{v}_k)_{k\in\mathcal{I}_J} \tag{2.11}$$

is defined by the property that

$$\sum_{j=1}^{J}\sum_{k\in\mathcal{I}_j} \mathbf{w}_{j,k}\varphi_{j,k} = \sum_{k\in\mathcal{I}_J} \mathbf{v}_k\varphi_{J,k}.$$

As an immediate consequence of Theorem 3, we obtain

$$\mathrm{cond}_2(\mathbf{H}^\intercal \mathbf{A}_y \mathbf{H}) \lesssim 1, \tag{2.12}$$

where $\mathbf{H}$ is often referred to as Bramble-Pasciak-Xu (BPX) preconditioner [6, 31].

Under mild conditions on the diffusion coefficients, these concepts lead to solvers for discretized elliptic problems within discretization error accuracy that require *linear time*, that is, the computational work is proportional to the system size. More precisely, this holds when discretization errors dominate the effect of floating point arithmetic (see, for example, the discussion in [28]).

2.4. **Effect of preconditioning on convergence of optimization schemes.** The observations made above on solvers for linear systems in essence carry over to the present context of optimization. In fact, using a well-conditioned representation also improves the convergence behaviour of optimization schemes for neural network approximations, as demonstrated in Table 1 (for the definitions of the error measures used, see Section 3.4). However, the same reservations regarding floating point precision as in the solver context apply, which is the issue addressed in the remainder of the paper.

TABLE 1. Comparison of FOSLS with and without preconditioning.

Optimizer	Precond.	Precision	RMSE of flux		RMSE of solution		MSE	Loss
			L_2-norm	H^1-norm	L_2-norm	$H(\mathrm{div})$-norm		
SGD	✘	`float32`	2.70×10^{-1}	8.99×10^{-1}	9.91×10^{-2}	3.48×10^{-1}	9.30×10^{-1}	7.46×10^{-1}
	✘	`float64`	2.70×10^{-1}	8.99×10^{-1}	9.91×10^{-2}	3.48×10^{-1}	9.30×10^{-1}	7.47×10^{-1}
	✓	`float32`	4.64×10^{-2}	4.78×10^{-2}	2.01×10^{-2}	8.74×10^{-2}	9.93×10^{-3}	1.06×10^{-2}
	✓	`float64`	4.62×10^{-2}	4.76×10^{-2}	2.01×10^{-2}	8.74×10^{-2}	9.91×10^{-3}	1.06×10^{-2}
Adam	✘	`float32`	4.31×10^{-2}	4.92×10^{-2}	2.38×10^{-2}	8.54×10^{-2}	9.72×10^{-3}	9.12×10^{-3}
	✘	`float64`	4.28×10^{-2}	4.79×10^{-2}	2.29×10^{-2}	8.34×10^{-2}	9.25×10^{-3}	8.83×10^{-3}
	✓	`float32`	3.59×10^{-4}	4.90×10^{-4}	1.40×10^{-4}	9.32×10^{-4}	1.11×10^{-6}	1.19×10^{-6}
	✓	`float64`	4.42×10^{-4}	5.26×10^{-4}	1.37×10^{-4}	8.45×10^{-4}	9.90×10^{-7}	1.14×10^{-6}
L-BFGS	✘	`float32`	4.31×10^{-2}	4.48×10^{-2}	2.05×10^{-2}	8.87×10^{-2}	9.87×10^{-3}	1.04×10^{-2}
	✘	`float64`	4.31×10^{-2}	4.43×10^{-2}	2.02×10^{-2}	8.35×10^{-2}	8.93×10^{-3}	9.34×10^{-3}
	✓	`float32`	1.64×10^{-3}	2.51×10^{-3}	7.50×10^{-4}	5.29×10^{-3}	3.43×10^{-5}	3.43×10^{-5}
	✓	`float64`	5.10×10^{-4}	8.48×10^{-4}	1.77×10^{-4}	1.45×10^{-3}	2.82×10^{-6}	2.93×10^{-6}
NGD	✘	`float32`	2.81×10^{-1}	9.40×10^{-1}	9.91×10^{-2}	3.33×10^{-1}	9.95×10^{-1}	8.05×10^{-1}
	✘	`float64`	2.81×10^{-1}	9.40×10^{-1}	9.91×10^{-2}	3.33×10^{-1}	9.95×10^{-1}	8.06×10^{-1}
	✓	`float32`	1.47×10^{-3}	1.68×10^{-3}	3.95×10^{-4}	2.93×10^{-3}	1.14×10^{-5}	1.20×10^{-5}
	✓	`float64`	1.08×10^{-3}	1.28×10^{-3}	2.62×10^{-4}	2.02×10^{-3}	5.70×10^{-6}	6.08×10^{-6}

2.5. **Effect on random initializations.** In the optimization over a set of neural network parameters θ in (2.5), also the random initialization of θ plays an important role. The network weights in θ are often chosen to be appropriately scaled i.i.d. samples of scalar Gaussian distributions. As illustrated in Figure 1, using such initializations to parameterize the coefficients of the frame representations in H^1 leads to starting values representing substantially more regular functions than with standard finite element basis functions. This means that when using the preconditioned form of the problem, the optimization starts from significantly lower loss values.

This observation can be interpreted in the context of representations of random fields: whereas linear combinations of standard finite element basis functions with i.i.d. standard Gaussian coefficients lead to approximations of white noise, combining such coefficients with frames in H^1 yields random fields having the regularity properties of Brownian motion. Although the random parameterization in terms of neural networks shown in Figure 1 (here for the full ResNet architecture as described in Section 3.2 with the initialization described above) is substantially more involved, we observe similar regularity of sample paths.

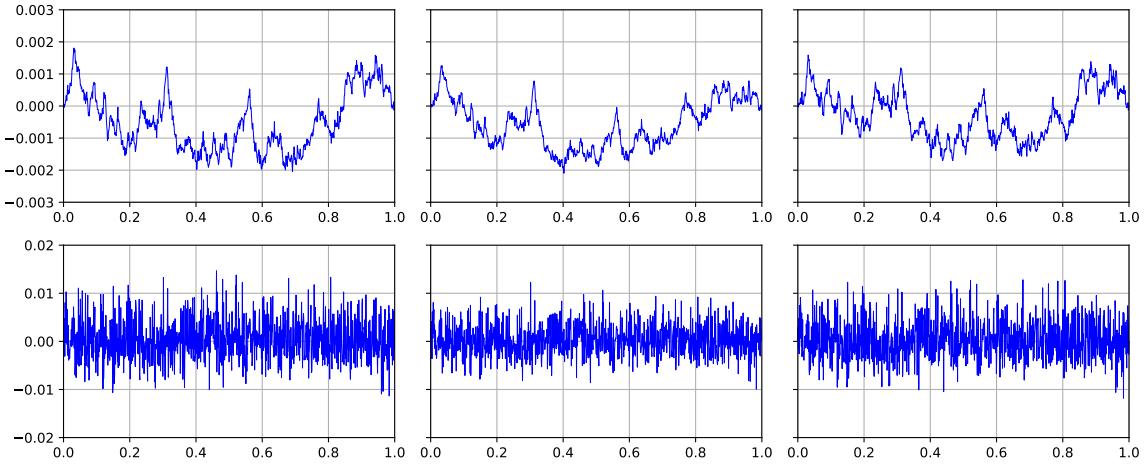

FIGURE 1. Samples of random initialization with and without frame representation. (**Top**) Initialization with H^1 finite element frame representation. (**Bottom**) Initialization without frame representation using standard finite element basis.

3. Numerical stability of preconditioned problems

Although the matrix $\mathbf{H}^\intercal \mathbf{A}_y \mathbf{H}$ considered above is well-conditioned, preconditioning in this form is in general not sufficient for numerical stability of methods, see also the discussion in [28]. While the convergence of iterative methods depending on the condition number may be improved, the achievable accuracy remains limited because when applied in this form, the action of the ill-conditioned matrices $\mathbf{A}_y$ and $\mathbf{H}$ still leads to a loss of precision. In this regard, standard recursive implementations of multilevel preconditioners (with linear costs) correspond to decomposing the well-conditioned symmetrically preconditioned matrix into the product $\mathbf{H}^\intercal \mathbf{A}_y \mathbf{H}$ of individually ill-conditioned matrices. However, we now consider how such ill-conditioned matrices can be completely avoided not only by operator preconditioning as proposed in [28], but also in the context of standard multilevel finite element bases.

In what follows, we assume a family of frames Φ_J in the trial space U with refinement parameter $J \in \mathbb{N}$ to be given such that for the corresponding constants $c_{\Phi_J}, C_{\Phi_J} > 0$ in (2.9),

$$\sup_{J \in \mathbb{N}} \frac{C_{\Phi_J}}{c_{\Phi_J}} < \infty. \tag{3.1}$$

As an example, we subsequently consider multilevel frames as in Section 2.3, where J corresponds to the maximum discretization level.

3.1. **Stable representations.** In what follows, suppressing the index J and thus writing $\Phi = \Phi_J$, for F as in (2.8), we write

$$H_\Phi = \operatorname{ran} \mathcal{D}F.$$

Note that since $\mathcal{D}$ is an operator of vector-valued weak derivatives, when $U_\Phi = \operatorname{span} \Phi \subset U$ is a (finite-dimensional) subspace of piecewise polynomials, then also H_Φ is a subspace of piecewise polynomials.

Proposition 4. *Let Φ satisfy (3.1) and assume that the variational formulation (2.2) is uniformly stable for $y \in Y$, that is,*

$$c_U \|u\|_U^2 \le \langle \mathcal{D}u, \mathcal{D}u\rangle_y \le C_U \|u\|_U^2 \quad \forall\, u \in U,\, y \in Y.$$

Then with constants independent of J, the following hold true:

(i) *$\mathcal{D}F\colon \ell_2(\mathcal{I}) \to H$ is uniformly well-conditioned as a mapping from $\ker(F)^\perp$ to H_Φ.*
(ii) *Moreover, in both (E) and (LS), the bilinear form $(h, \tilde h) \mapsto \langle h, \tilde h\rangle_y$ is bounded and elliptic on H_Φ uniformly in $y \in Y$.*

Proof. Regarding part (i), we have that for all $u, v \in \ker F^\perp$ we can bound the spectrum of $\mathcal{D}F$ as follows

$$\sup_{v\in\ker(F)^\perp} \frac{\langle \mathcal{D}Fv, \mathcal{D}Fv\rangle_y}{\|v\|_U^2} \le C_U \frac{\|Fv\|_U^2}{\|v\|_U^2} \le C_U \|F\|_{\ell_2\to U}^2 \le C_U C_{\Phi_J}^2.$$

Denoting by $P^\perp$ the U-projection onto $\ker(F)^\perp$, we have

$$\inf_{v\in U, \|v\|_U=1} \frac{\langle \mathcal{D}Fv, \mathcal{D}Fv\rangle_y}{\|v\|_U} \ge c_U \frac{\|FP^\perp v\|_U^2}{\|P^\perp v\|_U^2} \ge c_U c_{\Phi_J}^2.$$

Hence, we have $\operatorname{cond}((\mathcal{D}F)'\mathcal{D}F) \le \frac{C_U C_{\Phi_J}^2}{c_U c_{\Phi_J}^2}$.

Concerning (ii), in both cases (E) and (LS) we have $H_\Phi \subset \mathcal{D}(U)$, and uniform ellipticity is inherited under restriction to closed subspaces. Specifically, in case (LS), for all $h \in \mathcal{D}(U)$, by definition there exist $u \in H_0^1(\Omega)$ and $\tau \in H(\operatorname{div})$ such that $h = (\nabla u, \tau, \operatorname{div}(\tau))$. □

Now let $\{\chi_n\colon n \in \mathcal{N}\}$ with $\mathcal{N} = \mathcal{N}_J$, be a frame for H_Φ and define

$$\mathbf{C}_y = \big(\langle \chi_n, \chi_{n'}\rangle_y\big)_{n,n'\in\mathcal{N}}.$$

We assume that $G\colon H_\Phi \to \ell_2(\mathcal{N})$ is an injective mapping that is uniformly in J well-conditioned from H_Φ to $\operatorname{ran} G$ with the property

$$h = \sum_{n\in\mathcal{N}} (Gh)_n \chi_n \quad \text{for all } h \in H_\Phi.$$

As noted earlier, H_Φ is a space of piecewise polynomials, so that G can be obtained, for example, by piecewise polynomial interpolation.

Defining now

$$\mathbf{D} := G\mathcal{D}F,$$

from the above definitions we directly obtain

$$\mathbf{H}^\intercal \mathbf{A}_y \mathbf{H} = \mathbf{D}^\intercal \mathbf{C}_y \mathbf{D}.$$

The following is a direct consequence of Proposition 4 and the properties of G.

Corollary 5. *The condition numbers of $\mathbf{D}$ and $\mathbf{C}_y$ are bounded uniformly in J and $y \in Y$.*

Example 6. Consider (E) for $d = 1$ with $\Omega = (0,1)$. Let $\varphi_{j,k}(x) = 2^{-j}\max\{0, 1 - 2^j|x-k|\}$ for $k \in \mathcal{I}_j = \{1, \dots, 2^j - 1\}$ for $j \in \mathbb{N}$, so that $\{\varphi_{j,k}\colon j = 1, \dots, J,\, k \in \mathcal{I}_j\}$ is a frame for $H_0^1(\Omega)$ with constants independent of J as a consequence of Theorem 3. Let χ_n be the characteristic function of the interval $\Omega_n = 2^{-J}(n-1, n)$ for $n \in \mathcal{N}_J = \{1, \dots, 2^J\}$ and let

$x_n = 2^{-J}(n+\frac{1}{2})$. Then $\{\chi_n : n \in \mathcal{N}_J\}$ is a frame for H_Φ, which are piecewise constant functions on level J. We thus obtain

$$\mathbf{C}_y = \left(\delta_{n,n'} \int_{\Omega_n} a(x,y)\,dx\right)_{n,n'\in\mathcal{N}_J}, \quad \mathbf{D} = \left(\varphi'_{j,k}(x_n)\right)_{n\in\mathcal{N}_J,(j,k)}.$$

For $d > 1$ we can proceed analogously by choosing $\varphi_{j,k}$ as H^1-normalized piecewise linear finite elements on uniformly refined triangulations $\mathcal{T}_j$. Then we can choose $\{\chi_n : n \in \mathcal{N}_J\}$ as the basis of $\mathbb{P}_0(\mathcal{T}_J)^d$ formed from the characteristic functions of elements.

Example 7. For (LS) in the case $d = 1$ with $\Omega = (0,1)$, we have $H(\mathrm{div};\Omega) = H^1(\Omega)$. We can thus obtain a frame in $U = H^1_0(\Omega) \times H(\mathrm{div};\Omega) = H^1_0(\Omega) \times H^1(\Omega)$ by combining frames for the finite element spaces of level J in $H^1_0(\Omega)$ and $H^1(\Omega)$ analogously to Example (6). Here, evaluation of the bilinear form $\langle\cdot,\cdot\rangle_y$ requires integrals of products of piecewise linear functions with the coefficients a_y^{-1}. In case that the latter are piecewise constant on the finest grid of level J, for example, these can be resolved using evaluation in two Gauss points per element and constructing the frame of H_Φ by combining vector-valued piecewise constant and piecewise linear functions. In the case $d > 1$, to the best of our knowledge, the construction of uniformly stable multilevel frames of $H(\mathrm{div};\Omega)$, that are applicable to spaces of Raviart-Thomas finite elements on hierarchies of triangular meshes, remains open. This question will be investigated in a separate work.

3.2. **Network structures.** With a multilevel frame $\Phi = \Phi_J$ as in Section 2.3, we consider approximations of parameter-dependent solutions the form (1.1),

$$y \mapsto \sum_{j=1}^{J} \sum_{k\in\mathcal{I}_j} \mathbf{u}_{jk}(y;\theta)\varphi_{j,k},$$

where the vector $\mathbf{u}(\cdot;\theta)$ is represented by a neural network with trainable parameters θ.

We compare three different architectures, illustrated in Figure 2, for the neural networks mapping from the parameter space to the frame coefficients. A central element of all three architectures are the so-called *residual networks* (ResNets) [24]. These are widely used in scientific computing due to their favorable numerical stability properties.

We use residual networks with layers $\Psi_{(A,W,b)} : \mathbb{R}^n \to \mathbb{R}^n$, with n to be determined, of the particular form

$$z \;\mapsto\; \Psi_{(A,W,b)}(z) = z + A\sigma(Wz + b), \tag{3.2}$$

where $A \in \mathbb{R}^{n\times r}$, $W \in \mathbb{R}^{r\times n}$, $b \in \mathbb{R}^n$, with componentwise application of the activation function $\sigma : \mathbb{R} \to \mathbb{R}$. We will also call such $\Psi_{(A,W,b)}$ a ResBlock. Here, as in [2], we aim for $r \ll n$ to achieve a data-sparse representation, and by analogy to low-rank matrix representations, we will call r the *rank* of the layer.

Full low-rank ResNet: In the first architecture we use a fully connected layer

$$\mathbb{R}^{d_p} \ni p \mapsto \Lambda_0(p) = \sigma(W_0 p + b_0) \in \mathbb{R}^{\hat{N}},$$

mapping to the full frame space followed by a sequence of ResBlocks, that is,

$$\mathbf{u}(\cdot;\theta) = \Psi_{(A_L,W_L,b_L)} \circ \cdots \circ \Psi_{(A_2,W_2,b_2)} \circ \Psi_{(A_1,W_1,b_1)} \circ \Lambda_0.$$

Separate low-rank ResNet: In the second architecture, we exploit the multilevel structure of the frame itself by using a distinct ResNet with initial fully connected layer for each frame level. In the case (E), for the coefficients on each level j we use a separate ResNet representation of the form

$$\mathbf{u}_j(\cdot;\theta) = \Psi_{(A_{j,L_j},W_{j,L_j},b_{j,L_j})} \circ \cdots \circ \Psi_{(A_{j,2},W_{j,2},b_{j,2})} \circ \Psi_{(A_{j,1},W_{j,1},b_{j,1})} \circ \Lambda_0^j,$$

and $\mathbf{u}$ is obtained by concatenating the vectors $\mathbf{u}_j$ for $j \le J$. In (LS), we parameterize the solution components σ and u by two independent separate ResNets.

Separate frame representation: With $n_j = \#\mathcal{I}_j$, let the (sparse) prolongation matrices $P_j \in \mathbb{R}^{n_j \times n_{j-1}}$ be given for $j = 1, \dots, J$ by

$$\sum_{k \in \mathcal{I}_{j-1}} \mathbf{v}_k \varphi_{j-1,k} = \sum_{k \in \mathcal{I}_j} (P_j \mathbf{v})_k \varphi_{j,k}, \quad \mathbf{v} \in \mathbb{R}^{n_{j-1}}. \tag{3.3}$$

We define *prolongation layers* as $\Pi_{P_j} \colon \mathbb{R}^{n_{j-1}} \to \mathbb{R}^{n_j}$, $z \mapsto P_j z$.

Starting from a fully connected layer Λ_0 given by

$$\mathbb{R}^{d_p} \ni p \mapsto \Lambda_0(p) = \sigma(W_0 p + b_0) \in \mathbb{R}^{n_1},$$

the resulting neural networks are composed of layers $\Lambda_j \colon \mathbb{R}^{n_j} \to \mathbb{R}^{n_j}$ of the form

$$\Lambda_j = \Psi_{(A_{j,L_j}, W_{j,L_j}, b_{j,L_j})} \circ \cdots \circ \Psi_{(A_{j,2}, W_{j,2}, b_{j,2})} \circ \Psi_{(A_{j,1}, W_{j,1}, b_{j,1})} \circ \Pi_{P_j} \tag{3.4}$$

with some $L_j \in \mathbb{N}$, for $j = 1, \dots, J$. In case (E), for each level j, the composition of such $\Lambda_0, \dots, \Lambda_j$ parameterizes the frame coefficients $\mathbf{u}_j$ for this level,

$$\mathbf{u}_j(\cdot; \theta_j) = \Lambda_j \circ \cdots \circ \Lambda_1 \circ \Lambda_0,$$

with trainable parameters that are separate for each j,

$$\theta_j = (A^{(j)}_{j,L_j}, W^{(j)}_{j,L_j}, b^{(j)}_{j,L_j}, \dots, A^{(j)}_{j,1}, W^{(j)}_{j,1}, b^{(j)}_{j,1}, \dots, \\ \dots, A^{(j)}_{1,L_1}, W^{(j)}_{1,L_1}, b^{(j)}_{1,L_1}, \dots, A^{(j)}_{1,1}, W^{(j)}_{1,1}, b^{(j)}_{1,1}, W^{(j)}_0, b^{(j)}_0).$$

This corresponds to a levelwise application of the format used previously in [2]. In (LS), we again parameterize σ and u by two independent separate frame representations.

3.3. **Matrix-free numerical realization.** In the variational formulation (2.5), instead of assembling the stiffness matrix $\mathbf{A}_y$ explicitly, we employ a matrix-free evaluation of matrix-vector products. We again assume a multilevel frame $\Phi = \Phi_J$ as in Section 2.3 corresponding to the mesh $\mathcal{T}_J$ on level J. In particular, for $j = 1, \dots, J$, we have a basis $\{\varphi_{j,k} \colon k \in \mathcal{I}_j\}$ of the respective finite element space on level j.

We consider the variational formulation on the mesh $\mathcal{T}_J = \{T_1, \dots, T_{\#\mathcal{T}_J}\}$. For each element T_k, the local-to-global mapping is given by an index set

$$I_k := \{i_k^1, i_k^2, \dots, i_k^m\},$$

where m denotes the number of local degrees of freedom (e.g., $m = 2$ for one-dimensional linear elements and $m = 3$ for triangular elements in two dimensions). On this element, we define the local stiffness matrix

$$\mathbf{A}_y^k := \big(\langle \mathcal{D}\varphi_{J,\ell}, \mathcal{D}\varphi_{J,\ell'} \rangle_y\big)_{\ell,\ell' \in I_k} \in \mathbb{R}^{m \times m},$$

where $\mathcal{D}$ denotes the differential operator associated with the variational formulation.

Given a global solution vector $\mathbf{u} \in \mathbb{R}^{\#\mathcal{I}_J}$, we gather the corresponding local subvector

$$\mathbf{u}^k := (u_{i_k^1}, u_{i_k^2}, \dots, u_{i_k^m})^\intercal \in \mathbb{R}^m.$$

The local contributions are then assembled into the global vector via

$$\big(\mathbf{A}_y \mathbf{u}\big)_i = \sum_{k : i \in I_k} \big(\mathbf{A}_y^k \mathbf{u}^k\big)_{\mathrm{loc}(i, T_k)},$$

where $\mathrm{loc}(i, T_k)$ denotes the local index of the global degree of freedom i within element T_k, i.e., i is the $\mathrm{loc}(i, T_k)$-th entry of I_k. This matrix-vector product allows us to evaluate the variational formulation (2.5) without explicitly forming the global stiffness matrix.

Moreover, the elementwise products $\{\mathbf{A}_y^k \mathbf{u}^k\}_{k \in \mathcal{I}_J}$ can be computed in a vectorized manner. Specifically,

$$\widetilde{\mathbf{A}}_y := \begin{bmatrix} \mathbf{A}_y^1 \\ \vdots \\ \mathbf{A}_y^{\#\mathcal{I}_J} \end{bmatrix} \in \mathbb{R}^{\#\mathcal{I}_J \times m \times m}, \qquad \widetilde{\mathbf{u}} := \begin{bmatrix} \mathbf{u}^1 \\ \vdots \\ \mathbf{u}^{\#\mathcal{I}_J} \end{bmatrix} \in \mathbb{R}^{\#\mathcal{I}_J \times m},$$

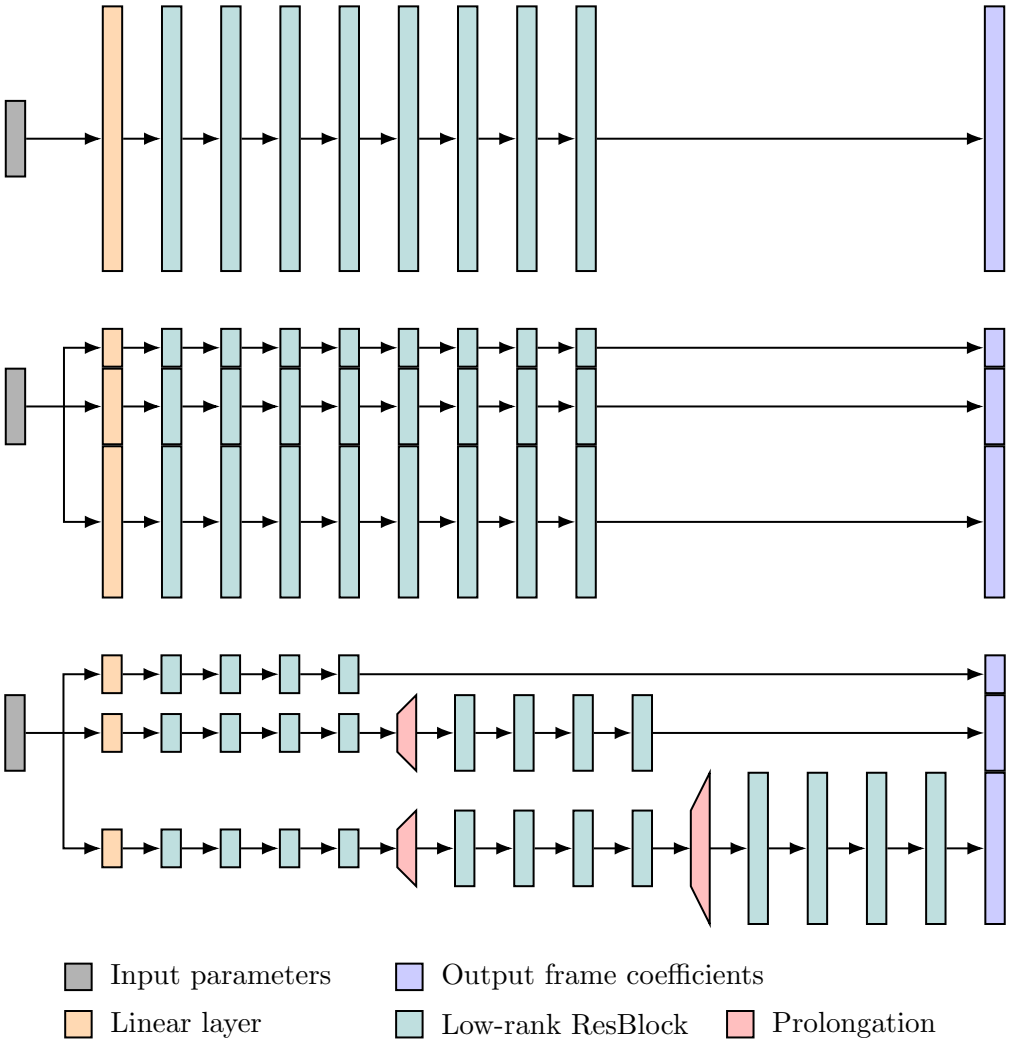

FIGURE 2. Different neural network architectures mapping from the parameter space to the frame coefficients. Without loss of generality, we take three-levels of frame as an example. **(Top)** Full low-rank ResNet. **(Middle)** Separate low-rank ResNet. **(Bottom)** Separate frame representation.

such that the product in each element can be computed all-at-once by a tensor contraction:

$$\left(\widetilde{\mathbf{A}}_y\right)_{k\alpha\beta}\left(\widetilde{\mathbf{u}}\right)_{k\alpha} = \begin{bmatrix} \mathbf{A}_y^1\mathbf{u}^1 \\ \vdots \\ \mathbf{A}_y^{\#\mathcal{I}_J}\mathbf{u}^{\#\mathcal{I}_J} \end{bmatrix} \in \mathbb{R}^{\#\mathcal{I}_J\times m}.$$

This formulation naturally extends to batched parameters y, enabling simultaneous evaluation of stiffness matrix-vector products for multiple parameter instances. Consequently, the matrix-free assembly procedure significantly reduces memory consumption and replaces sparse matrix-vector products by a batched dense tensor contraction, which are more efficient on GPUs.

For evaluating the product $\mathbf{Hw}$ for a given vector $\mathbf{w}$, we utilize the standard recursive implementation of multilevel preconditioners. Combining this with the above routines, yields a matrix-free numerical realization of the action of $\mathbf{H}^\intercal\mathbf{A}_y\mathbf{H}$.

Finally, we now turn to the matrix-free realization of the numerically stable decomposition $\mathbf{D}^\intercal\mathbf{C}_y\mathbf{D}$. Let w_n^q, x_n^q for $q = 1, \ldots, Q_n$ be Gauss quadrature weights and points corresponding to the triangle T_n such that quadratic functions are exactly integrated on the triangles. Suppose a_y is piecewise constant on this triangulation.

For a given coefficient vector $\mathbf{w}$, we introduce its subdivision into level components,

$$\mathbf{w} \coloneqq (\mathbf{w}_1, \dots, \mathbf{w}_J), \quad \mathbf{w}_j \in \mathbb{R}^{\#\mathcal{I}_j}.$$

Then

$$\mathcal{D}F(\mathbf{w}) = \mathcal{D}\left(\sum_{j=1}^{J} \sum_{k \in \mathcal{I}_j} w_{j,k} \varphi_{j,k} \right) = \sum_{j=1}^{J} \sum_{k \in \mathcal{I}_j} \mathbf{w}_{j,k} \mathcal{D}\varphi_{j,k}.$$

Consequently,

$$\begin{aligned} \langle \mathcal{D}F(\mathbf{w}), \mathcal{D}F(\mathbf{w}) \rangle_y &= \sum_{j,j'=1}^{J} \sum_{k,k' \in \mathcal{I}_j} \mathbf{w}_{j,k} \mathbf{w}_{j',k'} \langle \mathcal{D}\varphi_{j,k}, \mathcal{D}\varphi_{j',k'} \rangle_y \\ &= \sum_{j,j'=1}^{J} \sum_{k,k' \in \mathcal{I}_j} \mathbf{w}_{j,k} \mathbf{w}_{j',k'} \sum_{n=1}^{N_J} \sum_{q=1}^{Q} w_n^q a_y(x_n^q) \langle \mathcal{D}\varphi_{j,k}(x_n^q), \mathcal{D}\varphi_{j',k'}(x_n^q) \rangle_{\ell_2}. \end{aligned}$$

Therefore, the (q,n)-th row of the matrix $\mathbf{D}$ reads

$$\mathbf{D}_{q,n} \coloneqq \Big[\underbrace{\mathcal{D}\varphi_{1,1}(x_n^q), \dots, \mathcal{D}\varphi_{1,\#\mathcal{I}_1}(x_n^q)}_{\text{level } 1} \quad \cdots \quad \underbrace{\mathcal{D}\varphi_{J,1}(x_n^q), \dots, \mathcal{D}\varphi_{J,\#\mathcal{I}_J}(x_n^q)}_{\text{level } J} \Big].$$

In practice, it is not necessary to precompute or store the sparse matrix $\mathbf{D}$. Instead, one can evaluate the vector inner products on the fly,

$$\langle \mathbf{D}_{q,n}, \mathbf{w} \rangle_{\ell_2} = \sum_{j=1}^{J} \sum_{k \in \mathcal{I}_j} \Big\langle \underbrace{(\mathcal{D}\varphi_{j,k}(x_n^q), \dots, \mathcal{D}\varphi_{j,\#\mathcal{I}_j}(x_n^q))}_{\mathbf{D}_{q,n,j}}, \mathbf{w}_j \Big\rangle_{\ell_2},$$

which can be computed efficiently. Specifically, suppose that the quadrature point x_n^q lies in an element T_{j,k^*} at level j. The associated local-to-global index mapping is given by

$$I_{j,k^*} \coloneqq \{i_{k^*}^1, i_{k^*}^2, \dots, i_{k^*}^m\}.$$

We gather the corresponding local subvectors of $\mathbf{D}_{q,n,j}$ and $\mathbf{w}_j$, and compute their inner product. To vectorize this procedure, we apply the gather operator over all (q,n) simultaneously. A subsequent tensor contraction then yields the desired product $\mathbf{D}\mathbf{w}$.

3.4. **Experimental settings.** In our tests, we restrict ourselves to a one-dimensional parametric elliptic equation, since the effects that are relevant for this work do not differ in higher-dimensional cases. The diffusion field a_y is defined as

$$a_y = y_1 \chi_{[0,1/4)} + y_2 \chi_{[1/4,1/2)} + y_3 \chi_{[1/2,3/4)} + y_4 \chi_{[3/4,1]},$$

where $y \coloneqq (y_1, y_2, y_3, y_4) \in [0.5, 1.5]^4$, and χ_S denotes the characteristic function of the set S. The source term is set as a constant function $f \equiv 1$.

Metrics. As shown in (2.4), the FOSLS loss function (LS) provides a quantitative measure of the solution error. We compare it to other measures of accuracy for of the parameter-dependent coefficients $\mathbf{u}$, $\boldsymbol{\sigma}$ of the FOSLS formulation (LS) predicted by neural network-based surrogate models. These are evaluated on a test set of uniformly distributed parameters y^i, $i = 1, \dots, N_{\text{test}}$, using double precision (independently of the precision used for the training). The first are the *root mean square errors* (RMSE) of solution components with respect to the norm of a Hilbert space H on the spatial domain. For the coefficients $\mathbf{u}$ representing the function u, the RMSE in H-norm for N_{test} test samples is given by

$$\sqrt{\frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \|u(y^i) - u_{\text{FE}}(y^i)\|_H^2},$$

where u_{FE} is the exact finite element solution for the given parameter value and where we consider $H = L_2(\Omega)$ and $H = H^1(\Omega)$. For $\boldsymbol{\sigma}$ representing the function σ, we use the analogous expression with corresponding exact solution σ_{FE}, where norms here reduce to the same as for $\mathbf{u}$, but for the general case $d > 1$ would be those of $H = (L_2(\Omega))^d$ and

$H = H(\mathrm{div};\Omega)$. The second measure of accuracy is the total *mean squared error* (MSE) in U-norm, here given by

$$\frac{1}{N_{\mathrm{test}}}\sum_{i=1}^{N_{\mathrm{test}}}\left\{\|u(y^i)-u_{\mathrm{FE}}(y^i)\|^2_{H^1(\Omega)}+\|\sigma(y^i)-\sigma_{\mathrm{FE}}(y^i)\|^2_{H(\mathrm{div};\Omega)}\right\}.$$

Finally, as a consequence of (2.4), this quantity is proportional (with discretization-independent constants) to the FOSLS loss function as in (LS). For comparison, we evaluate this loss function in double precision on the test dataset. For frame coefficients predicted by the neural network, we first apply the frame synthesis operator and then substitute the reconstructed solution into the standard FOSLS loss.

Frame preconditioning. We employ piecewise linear elements for u and σ with $J = 10$ frame levels. The Lagrange basis functions are normalized with respect to the H^1-norm, which is computed using two-point Gaussian quadrature. When using half-precision arithmetic, in practice, we observe numerical instability when computing these H^1-norms. To address this issue, we precompute the H^1-norms in double precision and subsequently convert them to half or single precision for use during training in half or single precision.

Neural network architectures. We use the sigmoid linear unit (SiLU) [17, 30] activation function, and all network parameters are initialized using the Xavier Gaussian distribution [21].

(i) *Full low-rank ResNet.* For the full ResNet architecture, we employ 8 ResBlocks with a fixed rank of 8.

(ii) *Separate low-rank ResNet.* In the FOSLS setting, we parameterize σ and u using two independent ResNets. Each ResNet consists of 10 sub-ResNets, where each sub-ResNet maps the parameter space to the frame coefficients at a specific level. Every sub-ResNet contains 8 ResBlocks, and the rank of each ResBlock is set to the minimum of 8 and the input dimension of the ResBlock.

(iii) *Separate frame representations.* In the FOSLS framework, we also consider parameterizing σ and u using two independent frame-based representations. Between two consecutive prolongation operators, we insert 4 ResBlocks, i.e., $L_j = 4$ for each $j = 1,\dots,J$ in (3.4). The rank of the ResBlocks is chosen as the minimum of 8 and the input dimension of the ResBlock.

Hyper-parameters for training. We employ several optimizers depending on the experimental setup, with hyperparameters adjusted according to the float precision of computation (`float16`, `float32`, or `float64`). We use 6000 epochs for each optimizer.

(i) *Adam.* For the Adam optimizer, the initial learning rate η_0 is set to 10^{-3}. We apply a cosine annealing learning rate schedule

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\left(1+\cos\frac{t\pi}{T_{\max}}\right)$$

with period $T_{\max} = 5000$, and the minimum learning rate is set according to precision: 10^{-4} for half precision, 10^{-6} for single precision, and 10^{-10} for double precision. The parameter ϵ in Adam is chosen to ensure numerical stability: 10^{-4} for half precision, 10^{-8} for single precision, and 10^{-16} for double precision.

(ii) *SGD.* The standard SGD is combined with a cosine annealing learning rate schedule, whose hyperparameters are set as those of Adam.

(iii) *L-BFGS.* For the L-BFGS optimizer, we set the learning rate to 1.0, with a maximum of 20 iterations and 25 function evaluations per step. The gradient and parameter change tolerances are set according to the precision: for half precision, both are 10^{-4}; for single precision, 10^{-8}; and for double precision, 10^{-12}. The history size is fixed at 100, and the line search uses the strong Wolfe condition.

(iv) *Natural Gradient Descent (NGD).* For NGD, we use an initial step size of 1.0, combined with a line search satisfying the strong Wolfe conditions and allowing at most 20 line search steps. The inner linear system is solved using the conjugate gradient method

without explicitly forming the system matrix. The conjugate gradient solver is run for at most 20 iterations, and both the conjugate gradient tolerance and the ϵ regularization parameter depend on the numerical precision: 10^{-9} for half precision, 10^{-12} for single precision, and 10^{-14} for double precision.

3.5. **Numerically stable training.** As reported in Table 1, preconditioning leads to considerable improvements in the performance of neural network training even using the numerically unstable representation $\mathbf{H}^\intercal \mathbf{A}_y \mathbf{H}$ of the resulting system matrix. The corresponding loss curves are shown in Figure 3. However, the advantage of four orders of magnitude in the loss value is realized by the Adam optimizer, but not by SGD.

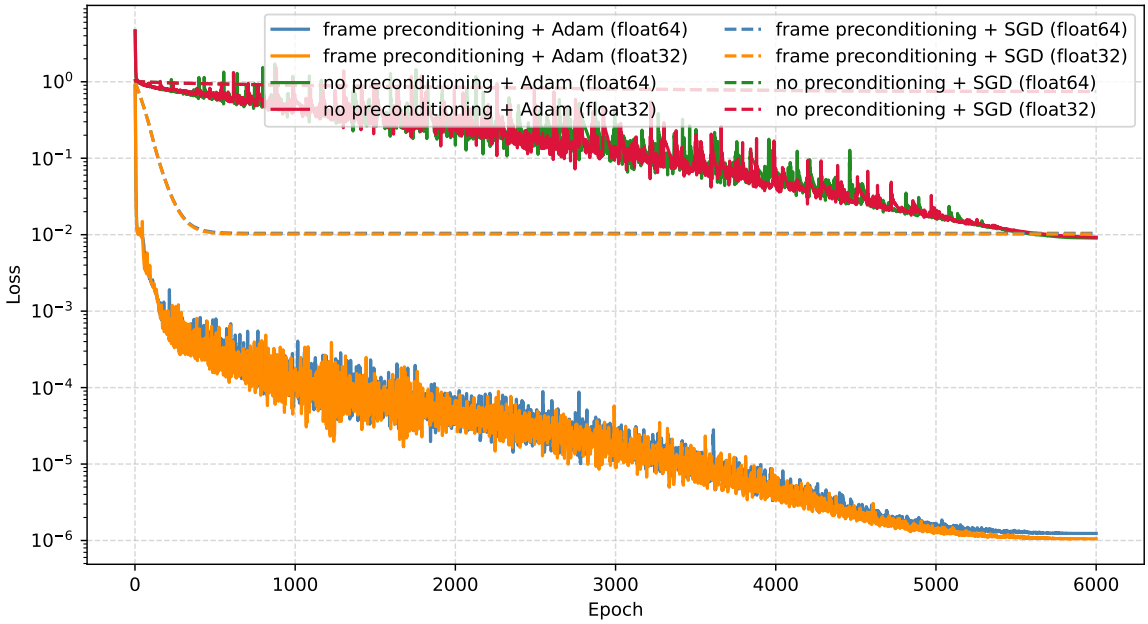

FIGURE 3. Loss curves of FOSLS with and without preconditioning using different optimizers under different computing precisions.

With the aid of stable frame representations, we can apply frame-based preconditioning also in half precision (using 16-bit floating point numbers), as reported in Table 2. Comparing Table 2 with Table 1, we observe that the accuracy is comparable in single and double precision. In half precision, the loss value achieved by Adam reaches 10^{-4}, which is close to the machine precision of half-precision floating-point arithmetic. Although L-BFGS and NGD perform well in single and double precision, their performance in half precision is inferior to that of Adam. In practice, we find that L-BFGS and NGD tend to perform better on shallower networks with larger rank when using half precision. It is worth noting that L-BFGS and NGD are more sensitive to numerical precision: their performance differs significantly between single and double precision, whereas the differences for SGD and Adam are relatively minor. The corresponding loss curves are shown in Figure 4.

We further compare different network architectures introduced in Section 3.2, as summarized in Table 3. The three neural networks have nearly the same number of trainable parameters (degrees of freedom, DoFs). All three architectures exhibit similar performance across the three numerical precisions.

4. Conclusions

In this work, we have investigated the effect of preconditioning on the training of neural networks for the approximation of parameter-dependent PDEs with focus on FOSLS formulations of elliptic problems. While compared to the formulation without preconditioning, we observe a striking improvement of training results for the Adam optimizer, this requires

TABLE 2. Comparison of FOSLS with stable preconditioning using different optimizers under different computing precisions.

Optimizer	Precision	RMSE of flux		RMSE of solution		MSE	Loss
		L_2-norm	H^1-norm	L_2-norm	$H(\mathrm{div})$-norm		
SGD	`float16`	5.55×10^{-2}	1.11×10^{-1}	2.56×10^{-2}	1.02×10^{-1}	2.28×10^{-2}	1.86×10^{-2}
	`float32`	4.64×10^{-2}	4.78×10^{-2}	2.01×10^{-2}	8.74×10^{-2}	9.93×10^{-3}	1.06×10^{-2}
	`float64`	4.62×10^{-2}	4.76×10^{-2}	2.01×10^{-2}	8.74×10^{-2}	9.91×10^{-3}	1.06×10^{-2}
Adam	`float16`	2.39×10^{-3}	7.53×10^{-3}	9.36×10^{-4}	6.95×10^{-3}	1.05×10^{-4}	1.02×10^{-4}
	`float32`	3.57×10^{-4}	4.93×10^{-4}	1.43×10^{-4}	9.41×10^{-4}	1.20×10^{-6}	1.13×10^{-6}
	`float64`	4.42×10^{-4}	5.25×10^{-4}	1.37×10^{-4}	8.44×10^{-4}	1.14×10^{-6}	9.88×10^{-7}
L-BFGS	`float16`	4.70×10^{-2}	6.02×10^{-2}	2.06×10^{-2}	9.13×10^{-2}	1.20×10^{-2}	1.21×10^{-2}
	`float32`	1.68×10^{-3}	2.17×10^{-3}	9.37×10^{-4}	6.28×10^{-3}	4.41×10^{-5}	4.31×10^{-5}
	`float64`	3.35×10^{-4}	4.09×10^{-4}	1.23×10^{-4}	9.39×10^{-4}	1.05×10^{-6}	1.19×10^{-6}
NGD	`float16`	1.17×10^{-2}	3.10×10^{-2}	4.07×10^{-3}	2.82×10^{-2}	1.76×10^{-3}	1.71×10^{-3}
	`float32`	1.15×10^{-3}	1.34×10^{-3}	3.63×10^{-4}	2.70×10^{-3}	9.09×10^{-6}	9.39×10^{-6}
	`float64`	9.73×10^{-4}	1.01×10^{-3}	2.73×10^{-4}	1.79×10^{-3}	4.22×10^{-6}	4.43×10^{-6}

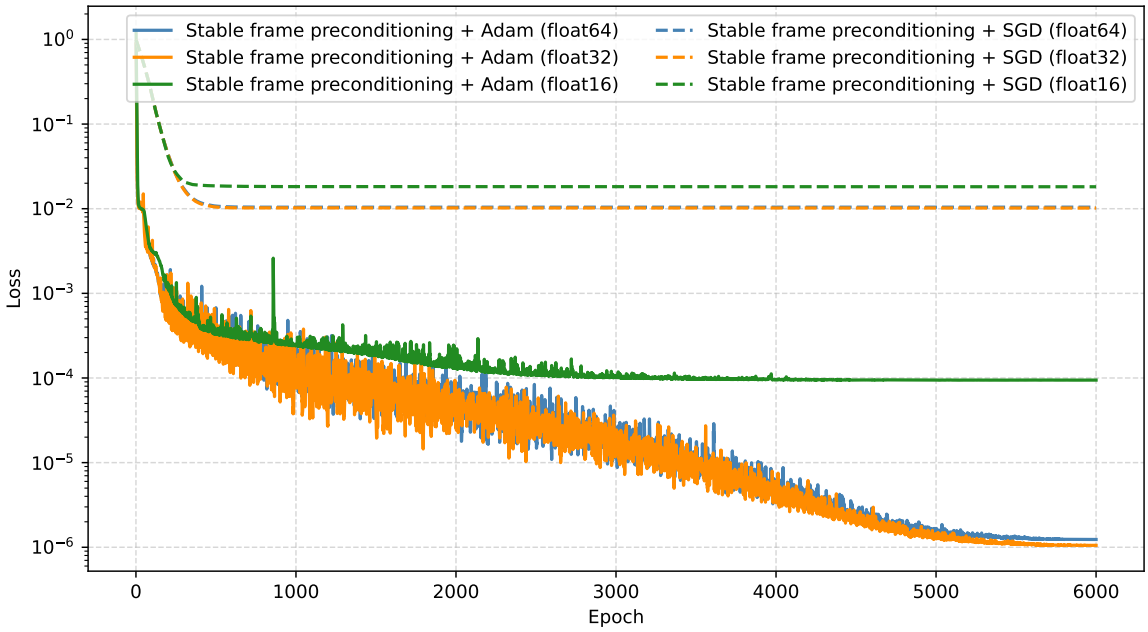

FIGURE 4. Loss curves of numerically stable preconditioning using different optimizers under different computing precisions.

TABLE 3. Comparison of FOSLS with stable preconditioning using different neural network architectures.

Architecture	DoFs	Precision	RMSE of flux		RMSE of solution		MSE	Loss
			L_2-norm	H^1-norm	L_2-norm	$H(\mathrm{div})$-norm		
Full ResNets	577036	`float16`	2.39×10^{-3}	7.53×10^{-3}	9.36×10^{-4}	6.95×10^{-3}	1.05×10^{-4}	1.02×10^{-4}
		`float32`	3.57×10^{-4}	4.93×10^{-4}	1.43×10^{-4}	9.41×10^{-4}	1.20×10^{-6}	1.13×10^{-6}
		`float64`	4.42×10^{-4}	5.25×10^{-4}	1.37×10^{-4}	8.44×10^{-4}	1.14×10^{-6}	9.88×10^{-7}
Sep. ResNets	577140	`float16`	2.94×10^{-3}	8.55×10^{-3}	8.76×10^{-4}	9.27×10^{-3}	1.59×10^{-4}	1.59×10^{-4}
		`float32`	1.08×10^{-3}	1.13×10^{-3}	1.15×10^{-4}	7.44×10^{-4}	2.32×10^{-6}	1.86×10^{-6}
		`float64`	1.41×10^{-3}	1.48×10^{-3}	1.29×10^{-4}	8.36×10^{-4}	3.56×10^{-6}	2.88×10^{-6}
Sep. frame rep.	551336	`float16`	6.86×10^{-3}	1.05×10^{-2}	1.79×10^{-3}	1.47×10^{-2}	3.29×10^{-4}	3.24×10^{-4}
		`float32`	1.08×10^{-3}	1.30×10^{-3}	6.12×10^{-4}	3.92×10^{-3}	1.63×10^{-5}	1.71×10^{-5}
		`float64`	1.28×10^{-3}	1.56×10^{-3}	4.86×10^{-3}	3.27×10^{-3}	1.29×10^{-5}	1.31×10^{-5}

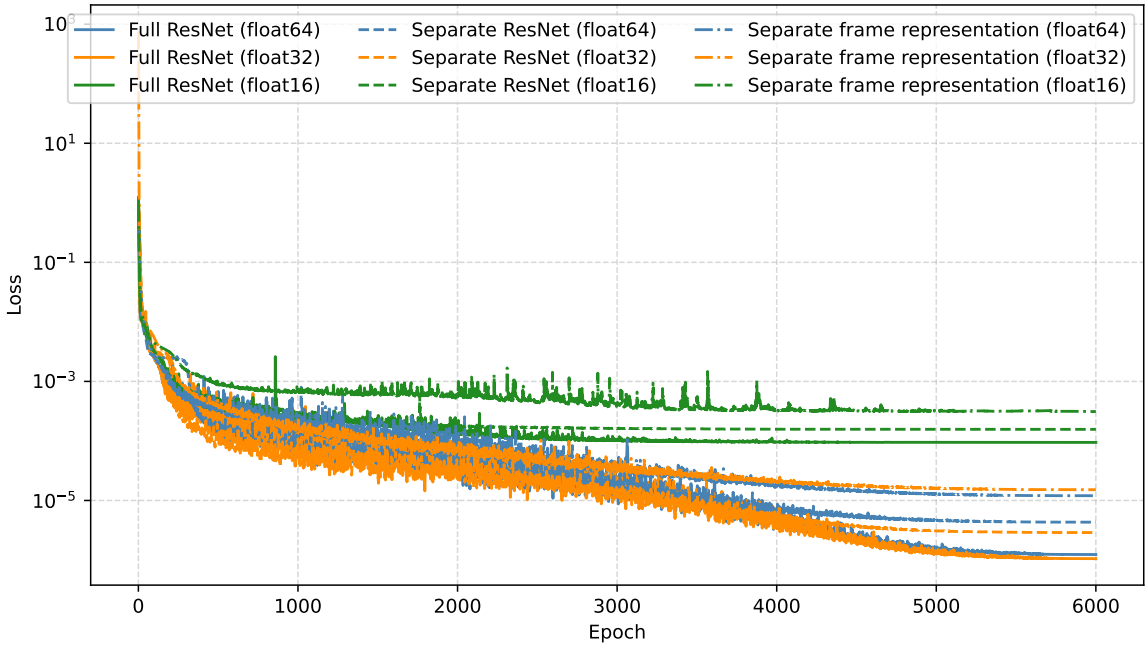

FIGURE 5. Loss curves of numerically stable preconditioning with different network architectures under different computing precisions.

working with sufficient machine precision. We have thus proposed a modified representation of preconditioned matrices that is still easy to implement efficiently that yields numerical stability and can be used for computations using half-precision floating point numbers.

Adam clearly outperforms SGD in all our tests with preconditioned problems. At the same time, methods such as L-BFGS or NGD do not yield any further improvement over Adam. The favorable optimization results obtained with Adam and preconditioning also show that the low-rank ResNet architecture adapted from [2] that we use can achieve total solution errors on the order of the discretization error expected for the given mesh size. Variants of this architecture that use separate representations for coefficients on different frame levels did not show an improvement over a simpler joint ResNet approximation of all frame coefficients.

ACKNOWLEDGEMENTS

The authors thank Polina Sachsenmaier and Konrad Westfeld for support with initial numerical tests.

Co-funded by the European Union (ERC, COCOA, 101170147). Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

REFERENCES

1. M. Bachmayr, W. Dahmen, C. Duan, and M. Oster, *Preconditioning and numerical stability in neural network training for parametric PDEs (Python code)*, Reproducibility archive, 10.5281/zenodo.18681906, 2026.
2. M. Bachmayr, W. Dahmen, and M. Oster, *Variationally correct neural residual regression for parametric PDEs: On the viability of controlled accuracy*, IMA Journal of Numerical Analysis (2025), draf073.
3. M. Bachmayr and V. Kazeev, *Stability of low-rank tensor representations and structured multilevel preconditioning for elliptic PDEs*, Found. Comput. Math. **20** (2020), no. 5, 1175–1236.
4. K. Bhattacharya, B. Hosseini, N. Kovachki, and A. Stuart, *Model reduction and neural networks for parametric PDEs*, SMAI-JCM: SMAI Journal of Computational Mathematics **7** (2021), 121–157.
5. P. B. Bochev and M. D. Gunzburger, *Least-squares finite element methods*, Applied Mathematical Sciences, vol. 166, Springer, New York, 2009. MR 2490235

6. J. H. Bramble, J. E. Pasciak, and J. Xu, *Parallel multilevel preconditioners*, Math. Comp. **55** (1990), no. 191, 1–22.
7. J. Bruna, B. Peherstorfer, and E. Vanden-Eijnden, *Neural Galerkin schemes with active learning for high-dimensional evolution equations*, Journal of Computational Physics **496** (2024), 112588.
8. Z. Cai, R. Lazarov, T. A. Manteuffel, and S. F. McCormick, *First-order system least squares for second-order partial differential equations: Part I*, SIAM Journal on Numerical Analysis **31** (1994), 1785–1799.
9. S. Cayci, *Gauss-Newton dynamics for neural networks: A Riemannian optimization perspective*, arXiv preprint: arXiv:2412.14031v3 [math.OC], 2024.
10. P. Cortés Castillo, W. Dahmen, and J. Gopalakrishnan, *DPG loss functions for learning parameter-to-solution maps by neural networks*, (2025).
11. W. Dahmen, *Wavelet and multiscale methods for operator equations*, Acta Numerica (1997), no. 6, 55–228.
12. W. Dahmen and A. Kunoth, *Multilevel preconditioning*, Numer. Math. **63** (1992), 315–344.
13. W. Dahmen, W. Li, Y. Teng, and Z. Wang, *Expansive natural neural gradient flows for energy minimization*, (2025).
14. W. Dahmen and R. Schneider, *Composite wavelet bases for operator equations*, Math. Comp. (1999), no. 68, 1533–1567.
15. W. Dahmen and R. Stevenson, *Element-by-element construction of wavelets satisfying stability and moment conditions*, SIAM Journal on Numerical Analysis (1999), no. 37, 319–325.
16. M. Deiml and D. Peterseim, *Quantum realization of the finite element method*, Mathematics of Computation (2025).
17. S. Elfwing, E. Uchibe, and K. Doya, *Sigmoid-weighted linear units for neural network function approximation in reinforcement learning*, Neural Networks **107** (2018), 3–11, Special issue on deep reinforcement learning.
18. T. Führer, R. Gonzales, and M. Karkulik, *Well-posedness of first order acoustic wave equations and space-time finite element approximation*, arXiv preprint arXiv:2311.10536, 2023.
19. T. Führer and M. Karkulik, *Space-time least-squares finite elements for parabolic equations*, Comput. Math. Appl. **92** (2021), 27–36.
20. M. Geist, P. Petersen, M. Raslan, R. Schneider, and G. Kutyniok, *Numerical solution of the parametric diffusion equation by deep neural networks*, Journal of Scientific Computing **88** (2021), no. 1, 22.
21. X. Glorot and Y. Bengio, *Understanding the difficulty of training deep feedforward neural networks*, Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, Proceedings of Machine Learning Research, vol. 9, PMLR, 2010, pp. 249–256.
22. F. Griese, F. Hoppe, A. Rüttgers, and P. Knechtges, *Preconditioned fem-based neural networks for solving incompressible fluid flows and related inverse problems*, Journal of Computational and Applied Mathematics **469** (2025), 116663.
23. H. Harbrecht, R. Schneider, and C. Schwab, *Multilevel frames for sparse tensor product spaces*, Numerische Mathematik **110** (2008), no. 2, 199–220.
24. K. He, X. Zhang, S. Ren, and J. Sun, *Deep residual learning for image recognition*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June 2016.
25. N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar, *Neural operator: Learning maps between function spaces with applications to PDEs*, Journal of Machine Learning Research **24** (2023), 1–97.
26. G. Kutyniok, P. Petersen, M. Raslan, and R. Schneider, *A theoretical analysis of deep neural networks and parametric PDEs*, Constructive Approximation **55** (2022), no. 1, 73–125.
27. W. Li and G. Montufar, *Natural gradient via optimal transport*, Information Geometry **1** (2018), 181–214.
28. S. Mohr, Y. Nakatsukasa, and C. Urzúa-Torres, *Full operator preconditioning and the accuracy of solving linear systems*, IMA J. Numer. Anal. **44** (2024), no. 6, 3259–3279.
29. P. Oswald, *Multilevel finite element approximation: Theory and applications*, Teubner Skripten zur Numerik, 1994.
30. P. Ramachandran, B. Zoph, and Q. V. Le, *Swish: a self-gated activation function*, 2017, arXiv:1710.05941.
31. J. Xu, *Iterative methods by space decomposition and subspace correction*, SIAM Review **34** (1992), no. 4, 581–613.